

I'm going to treat this as a product-spec pass, not a coding pass yet. I'll separate true MVP requirements from "power features" so we don't build a monster that never ships.

Thought for 46s >

My honest take

You are right about the direction. **Prompt Refinery should become a prompt/mode/context/loop operating system**, not just a prompt rewriter.

But you are wrong if every idea becomes an MVP requirement. That is how the product gets bloated and never becomes usable. The MVP should prove this core loop:

```
User input → context pack → mode → questions → canonical output → critique → next p
```

Everything else should plug into that loop later.

The current app already has the seed of this: it has `Prompt Output`, `Refined Response`, and `Custom Mode` configs in `App.tsx`, plus a `makeCustomMode()` function that turns raw text into a reusable custom mode config. The new PR #3 starts separating that into a proper typed Mode Manager with built-in/custom modes.

1. The big product idea

Prompt Refinery should have **five controllable layers**:

Layer	What it controls
Families	What kind of output is being created
Modes	How the AI behaves while creating it
Context Packs	What files/images/videos/thread history are included
Loops	How many passes happen and what triggers each pass
Connectors	Where the prompt/output goes next: local model, OpenRouter, n8n, webhook, MCP-style tool, etc.

That gives you a clean mental model:

```
Family = output type
Mode = behavior
Context = evidence
```

```
Loop = process
Connector = destination
```

2. Families that should exist in `prompt_library/`

These are the folders I would eventually add.

Core writing / thinking families

```
analysis
research_brief
summary
critique_loop
decision_memo
comparison_matrix
argument_map
study_guide
teaching_tutor
explanation
fact_check
source_audit
```



Product / coding / builder families

```
product_spec
mvp_spec
technical_spec
architecture_plan
execution_plan
implementation_plan
debugging_troubleshooting
code_review
repo_audit
refactor_plan
test_plan
qa_checklist
deployment_plan
api_spec
database_schema
frontend_ui_spec
backend_service_spec
devops_runbook
```



Agent / workflow families

```
agent_spec
skill_spec
tool_spec
workflow_spec
loop_spec
mcp_server_spec
webhook_spec
connector_spec
automation_blueprint
n8n_workflow_brief
make_scenario_brief
zapier_zap_brief
human_in_the_loop_spec
```



Prompt-engineering families

```
system_prompt
developer_prompt
user_prompt
few_shot_examples
prompt_wrapper
prompt_chain
prompt_router
prompt_classifier
prompt_evaluator
prompt_optimizer
token_reduction
context_compression
```



Business / freelance / content families

```
proposal_package
fiverr_gig
job_application
outreach_persuasion
sales_copy
landing_page_copy
youtube_script
shorts_script
social_post
email_sequence
brand_voice
customer_support_reply
```



Personal productivity families

```
daily_plan
weekly_review
goal_breakdown
habit_loop
journal_prompt
learning_plan
career_plan
resume_bullet
interview_prep
```



Data / document families

```
ocr_cleanup
document_extraction
spreadsheet_plan
csv_schema
json_schema
data_cleaning_plan
report_generation
pdf_summary
file_inventory
```



3. Custom modes that should exist

Modes are not the same as families. A mode is the operating behavior.

Built-in modes

```
minimal_token_efficiency
one_night_ai_stack_builder
deep_research
fast_answer
step_by_step_teacher
debugging_lead
product_manager
senior_engineer
strict_mvp
critique_first
ask_questions_first
no_questions_best_assumption
portfolio_builder
freelancer_proposal
local_only_privacy
api_cost_guard
high_accuracy_citation
creative_exploration
```



Custom mode fields

Every custom mode should have frontmatter like this:

YAML



```
mode_id: custom_mode_slug
display_name: Custom Mode Name
version: 1
status: custom
mutable: true
copyable: true
parent_mode_id: minimal_token_efficiency
default_prompt_family: execution_plan
token_policy: minimal | focused | normal | deep
scope_policy: exact_task_only | smallest_next_action | strict_mvp | open_exploration
question_policy: ask_blocking_only | ask_first | assume_and_continue
critique_policy: optional | required_before_accept | auto_generate
context_policy: none | thread_summary | selected_files | full_context_pack
loop_policy: none | manual | semi_auto | auto
connector_policy: none | local_model | openrouter | webhook | mcp_style
```

Mode controls the UI should expose

```
Mode name
Description
Default family
Token policy
Scope policy
Question policy
Critique policy
Context policy
Loop policy
Provider / connector policy
Stop conditions
Max passes
Max estimated tokens
Allowed tools
Blocked tools
Output format
Next-prompt behavior
Thread-close behavior
```



4. Yes: Loop Mode should exist

Loop Mode is one of the most important features.

But it needs to start simple. Do **manual loop mode first**, then semi-auto, then API-connected loop mode.

Loop Mode definition

Loop Mode = a controlled multi-pass workflow where each pass has:
input → context → model/provider → output → critique → decision → next prompt



Loop types

Loop type	Meaning
Manual loop	App gives next prompt. User copies/pastes into Codex/ChatGPT/etc.
Semi-auto loop	App sends to configured endpoint, but user approves each next pass.
Auto loop	App runs until stop condition, budget, or success check.
Human-review loop	Every pass pauses for accept/defer/fix decision.
Webhook loop	Each pass can call an n8n/Make/generic webhook.
MCP-style loop	App exposes/uses resources, prompts, tools, and user elicitation.

MCP is directly relevant here. Official MCP defines server features as **resources**, **prompts**, and **tools**, and client features including **sampling**, **roots**, and **elicitation**. It also uses JSON-RPC 2.0 and has stateful connections/capability negotiation.

Model Context ...

Loop Mode config

</> YAML



```
loop_id: codex_reviewer_loop
display_name: Codex Reviewer Loop
mode_id: strict_mvp
family: execution_plan
max_passes: 5
max_tokens_estimate: 12000
stop_when:
  - user_accepts
  - no_open_questions
  - critique_passes
  - max_passes_reached
```

```
on_each_pass:
  - summarize_thread
  - estimate_tokens
  - generate_next_prompt
  - ask_close_or_continue
approval_required:
  - before_external_call
  - before_tool_call
  - before_webhook
  - before_loop_continue
```

5. Endpoint / connector support

Yes, you can support Ollama, OpenRouter, local endpoints, n8n, Make, webhooks, and MCP-style connections.

Provider connector types

```
local_ollama
openrouter
openai_compatible
gemini
anthropic
custom_http
n8n_webhook
make_webhook
zapier_webhook
mcp_style_server
file_system
github_repo
browser_manual_copy
```



Provider adapter shape

📄 TypeScript



```
type ProviderAdapter = {
  id: string;
  label: string;
  kind:
    | "ollama"
    | "openrouter"
    | "openai_compatible"
    | "webhook"
    | "mcp_style"
    | "manual_copy";
  baseUrl?: string;
```

```

apiKeyRef?: string;
model?: string;
supportsStreaming: boolean;
supportsVision: boolean;
supportsFiles: boolean;
supportsTools: boolean;
supportsJsonMode: boolean;
maxInputTokens?: number;
maxOutputTokens?: number;
};

```

Ollama

Ollama is a perfect first local provider because its API defaults to:

`http://localhost:11434/api`



The official Ollama API docs list endpoints including generate, chat, embed, tags, model details, pull, push, delete, and version.  [Ollama Docum...](#)

OpenRouter

OpenRouter is a good hosted/free-model provider because it normalizes many providers through a schema similar to the OpenAI Chat API, and its chat completions endpoint is

`/api/v1/chat/completions`.

 [OpenRou...](#)

n8n / webhook

n8n is a strong automation connector. Its Webhook node can trigger workflows, receive data, and return workflow results, making it usable like an API endpoint. n8n also has separate test and production webhook URLs and supports common HTTP methods plus basic/header/JWT/no auth options.  [n8n D...](#)  [n8n D...](#)

Make

Make custom webhooks can receive external HTTPS requests and trigger scenarios immediately or on a schedule; Make also supports queueing and rate-limit behavior for webhooks.  [make.c...](#)

MCP-style

I would not build full MCP first. Build an **MCP-shaped internal connector model** first:

```

resources = context files, summaries, images, transcripts
prompts = reusable templates
tools = callable actions
roots = allowed workspace boundaries

```




```
elicitation = question popups
sampling = asking another model/provider to generate
```

MCP's official safety guidance matters here: user consent, control, data privacy, tool safety, and approval before tool invocation are explicit security principles. [Model Context ...](#)

6. Context feature: yes, absolutely

This should be a major MVP feature, but it needs boundaries.

Context Pack system

Context Pack = selected evidence added to a run

Context types:

```
thread history
thread summary
files
images
video transcript
audio transcript
repo files
webpage text
manual notes
accepted prompts
saved roles
previous critiques
previous decisions
```

Context Pack metadata

`</>` YAML

```
context_pack_id: ctx_0001
name: Fiverr Gig Rewrite Context
sources:
  - type: file
    path: uploads/gig_notes.md
  - type: image
    path: uploads/thumbnail.png
  - type: thread_summary
    path: conversations/0000/summary.md
include_policy: selected_only
privacy_level: local_only
```

```
max_tokens: 8000
compression: summary_first
```

Context UI

The app needs a right-side or popup panel:

```
Attach context
Select files
Select images
Paste video/audio transcript
Add manual notes
Show estimated tokens
Compress context
Include / exclude source
Preview context pack
```



For video, do not start with raw video analysis. Start with:

```
video file → transcript/description → context pack
```



Images can start as uploaded image metadata + user description, then later vision model support.

7. Thread closing / summary / token saving

Yes. This should be in MVP.

Every thread should end with a **Close Thread** panel:

```
What was done
Final accepted output
Open questions
Decisions made
Files/context used
Next action
Suggested next prompt
Estimated token usage
Recommendation: close or continue
```



Close-thread prompt template

```
Summarize this thread into a compact continuation memory.
```



```
Return:
```

1. User goal
2. Final accepted output
3. Decisions made
4. Open questions
5. Important constraints
6. Files/context used
7. Next action
8. Exact next prompt
9. What can be safely omitted next time

Token-saving suggestion

The app should say:

This thread is getting long. Close it and continue from a summary to reduce token u 

This is a great product feature because it teaches the user how to preserve progress without bloating context.

8. Token estimate after every action

Yes, but do not pretend it is exact unless you use tokenizer libraries per model.

For MVP, call it:

Estimated token load 

Not:

Exact tokens used 

Token panel

Input estimate 
Context estimate
Output estimate
Total run estimate
Budget remaining
Close-thread recommendation
Cheaper mode suggestion

Token policies

```
minimal
focused
normal
deep
research_heavy
vision_heavy
loop_budged
```



9. Question / critique popup system

Yes. This should become one of the central UX patterns.

Question popup controls

Each question should have:

```
Answer
Skip
Use best assumption
Defer
Turn into task
Mark as blocking
Mark as non-blocking
Attach context to answer
```



Critique popup controls

Each critique should have:

```
Accept fix
Reject
Defer
Apply partially
Turn into next prompt
Turn into checklist item
Regenerate critique
Mark resolved
```



Critique template type

YAML



```
critique_id: scope_guard_v1
severity: low | medium | high
category: clarity | scope | correctness | token_cost | missing_context | risk
action:
```

```
- fix_now
- defer
- ignore
- convert_to_next_prompt
```

This becomes the “human-in-the-loop” core of the product.

10. Wrappers we should add

Wrappers are reusable operating shells.

AI provider wrappers

```
openrouter_chat_wrapper
ollama_local_chat_wrapper
openai_compatible_wrapper
gemini_flash_wrapper
anthropic_messages_wrapper
generic_http_llm_wrapper
```



Coding wrappers

```
codex_activation_prompt
codex_task_wrapper
codex_review_response_wrapper
codex_next_prompt_wrapper
cursor_task_wrapper
github_pr_review_wrapper
repo_audit_wrapper
```



Workflow wrappers

```
n8n_webhook_wrapper
make_webhook_wrapper
zapier_webhook_wrapper
generic_webhook_wrapper
mcp_style_tool_wrapper
manual_copy_loop_wrapper
```



Context wrappers

```
file_context_pack_wrapper
image_context_pack_wrapper
video_transcript_context_wrapper
repo_context_wrapper
```



```
thread_summary_wrapper  
source_citation_wrapper
```

Safety / cost wrappers

```
token_budget_wrapper  
privacy_local_only_wrapper  
api_cost_guard_wrapper  
tool_permission_wrapper  
webhook_consent_wrapper  
loop_stop_condition_wrapper
```



11. What the app currently needs that it does not include yet

Based on the repo state and the PR we just opened, the app needs these next:

Immediate missing pieces

```
Mode Manager mounted into Step 0  
Active mode state connected to refinery engine  
Custom mode persistence  
Mode import/export  
Mode frontmatter parser  
Context Pack panel  
Thread close summary  
Token estimate panel  
Question popup actions  
Critique popup actions  
Loop Mode config  
Provider connector registry  
Webhook connector config  
Provider secrets handling  
Safety/consent gate
```



Important technical debt

```
Current ModeManager is standalone, not mounted  
Mode templates are static TypeScript, not loaded from prompt_library files  
Custom mode duplicates are browser-state only  
No secure credential storage  
No provider abstraction yet  
No file/image/video context pipeline yet  
No token estimator  
No loop runner
```



No connector logs
No permissions model

12. MVP phases and passes

I would define MVP as complete when Prompt Refinery can:

Create prompts, use modes, attach context, critique outputs, summarize/close thread



That is enough to be useful before full automation.

Phase 1 — Mode Manager MVP

Goal: make modes real in the UI.

Pass	What it does
1.1	Mount <code>ModeManager</code> into Step 0
1.2	Replace old custom-mode controls with built-in/custom tabs
1.3	Connect selected mode to <code>refinePrompt()</code>
1.4	Duplicate built-in mode to custom
1.5	Save custom modes locally
1.6	Import/export custom mode Markdown

Done when: user can choose Minimal Token Efficiency or One-Night Builder, duplicate it, edit it, and use it in a run.

Phase 2 — Template Registry MVP

Goal: make `prompt_library/` actually power the app.

Pass	What it does
2.1	Add frontmatter parser
2.2	Load template metadata from static registry
2.3	Show families in UI

Pass	What it does
2.4	Match family to mode
2.5	Add template preview
2.6	Add template copy/export

Done when: templates are not just files; the app can show and apply them.

Phase 3 — Context Pack MVP

Goal: add controlled context.

Pass	What it does
3.1	Add Context panel
3.2	Add manual notes context
3.3	Add file text context
3.4	Add image description context
3.5	Add thread summary context
3.6	Add context token estimate
3.7	Add include/exclude toggles

Done when: user can attach context and see what will be included before generation.

Phase 4 — Questions / Critiques UX

Goal: make questions and critiques feel like a workflow, not random text.

Pass	What it does
4.1	Question popup component
4.2	Answer / skip / assume / defer actions
4.3	Critique popup component

Pass	What it does
4.4	Fix / defer / reject / convert to next prompt
4.5	Critique status tracking
4.6	Acceptance gate checks unresolved critiques

Done when: every question and critique has a decision state.

Phase 5 — Thread Summary and Token Saver

Goal: prevent context bloat.

Pass	What it does
5.1	Add close-thread button
5.2	Generate thread summary
5.3	Save summary to conversation folder
5.4	Suggest close/continue based on estimated tokens
5.5	Create continuation prompt
5.6	Reopen from summary

Done when: user can close a thread and continue later without dragging the whole conversation forward.

Phase 6 — Manual Loop Mode MVP

Goal: loop behavior without risky automation.

Pass	What it does
6.1	Add Loop Mode config type
6.2	Add max passes / stop conditions
6.3	Add pass history

Pass	What it does
6.4	Add next prompt generator
6.5	Add human approval before next pass
6.6	Add loop summary after each pass

Done when: the app can guide repeated Codex/ChatGPT/Perplexity passes manually.

At this point, I would call the MVP finished.

13. Post-MVP phases

Phase 7 — Provider Connectors

```
Ollama local
OpenRouter
OpenAI-compatible custom endpoint
Generic HTTP endpoint
Provider test button
Provider logs
Provider error handling
```



Phase 8 — Webhook / Automation Connectors

```
n8n webhook connector
Make webhook connector
Generic webhook connector
Webhook auth
Webhook payload template
Webhook response parser
Webhook run logs
```



Phase 9 — Semi-Auto Loop Mode

```
Send prompt to endpoint
Receive response
Auto-generate critique
Ask user to approve next pass
Track budget
Stop at pass limit
```



Phase 10 — MCP-style Connector Layer

```
Resources
Prompts
Tools
Roots
Elicitation
Tool permission UI
Tool logs
Tool safety checks
```



Do not skip safety here. MCP-style tool access is powerful, but official MCP docs explicitly emphasize user consent, data privacy, and tool safety because tools can represent arbitrary code execution.  Model Context ...

Phase 11 — Multi-modal Context

```
Image uploads
PDF extraction
Audio transcript
Video transcript
Screenshot context
Vision model provider support
Context compression
```



Phase 12 — Productization

```
User accounts
Projects
Cloud sync
Template marketplace
Team sharing
Billing
Usage analytics
Version history
Plugin system
```



14. Agents, skills, specs, and scope

You should add these folders eventually:

```
agents/
skills/
specs/
scopes/
connectors/
loop_modes/
```



```
context_packs/  
evaluators/  
wrappers/
```

agents/

Agent templates:

```
senior_engineer_agent  
product_manager_agent  
debugging_agent  
research_agent  
copywriter_agent  
workflow_automation_agent  
repo_reviewer_agent  
prompt_optimizer_agent
```



skills/

Skill templates:

```
summarize_thread  
compress_context  
estimate_tokens  
classify_prompt_family  
generate_questions  
critique_output  
generate_next_prompt  
create_loop_plan  
call_webhook  
parse_provider_response
```



specs/

Spec templates:

```
product_spec  
technical_spec  
mode_spec  
loop_spec  
connector_spec  
context_pack_spec  
security_spec  
provider_adapter_spec  
webhook_payload_spec  
mcp_style_spec
```



scopes/

Scope templates:

```
strict_mvp_scope
one_night_scope
portfolio_demo_scope
client_delivery_scope
local_only_scope
free_api_scope
research_deep_scope
```



15. The prompt templates I would add first

Here is the high-value first library.

```
prompt_library/
  analysis/
    template_v1.md
    source_audit_v1.md
    comparison_matrix_v1.md
    decision_memo_v1.md

  product_spec/
    template_v1.md
    mvp_spec_v1.md
    user_story_map_v1.md
    feature_scope_v1.md

  execution_plan/
    template_v1.md
    one_night_build_plan_v1.md
    vertical_slice_plan_v1.md
    dependency_order_plan_v1.md

  debugging_troubleshooting/
    template_v1.md
    error_triage_v1.md
    minimal_repro_v1.md
    patch_plan_v1.md

  critique_loop/
    template_v1.md
    scope_critique_v1.md
    token_cost_critique_v1.md
    correctness_critique_v1.md
    missing_context_critique_v1.md

  loop_spec/
    manual_loop_v1.md
```



```
semi_auto_loop_v1.md
codex_reviewer_loop_v1.md
provider_loop_v1.md

context_pack/
  file_context_v1.md
  image_context_v1.md
  thread_summary_context_v1.md
  repo_context_v1.md

connectors/
  ollama_connector_v1.md
  openrouter_connector_v1.md
  n8n_webhook_connector_v1.md
  make_webhook_connector_v1.md
  generic_webhook_connector_v1.md
  mcp_style_connector_v1.md

wrappers/
  codex_task_wrapper_v1.md
  perplexity_planning_wrapper_v1.md
  ollama_local_wrapper_v1.md
  openrouter_free_model_wrapper_v1.md
  webhook_payload_wrapper_v1.md
```

16. MVP clarification questions

These are the only questions that matter right now:

1. Should MVP be local-only first, or can it call OpenRouter/n8n from the app?

My recommendation: local-only/manual loop first, then OpenRouter/webhooks.

2. Should custom modes save to local files or browser storage first?

My recommendation: local file-writer API, because your repo is already built around saving artifacts.

3. Should file/image/video context be included in MVP?

My recommendation: text files and manual image descriptions in MVP; raw video later.

4. Should Loop Mode execute automatically?

My recommendation: no. MVP should be manual approval after every pass.

5. What is the first real use case to test everything?

My recommendation: `Codex project builder loop` because it uses modes, context, critiques, next prompts, summaries, and token-saving.

Final recommendation

Build the MVP in this order:

1. Mode Manager mounted
2. Custom mode save/edit/duplicate
3. Template registry
4. Context Pack panel
5. Question + Critique popups
6. Thread summary + close-thread
7. Token estimate panel
8. Manual Loop Mode



Then call MVP done.

After that:

9. Ollama connector
10. OpenRouter connector
11. Generic webhook connector
12. n8n/Make connector presets
13. Semi-auto loop mode
14. MCP-style resources/prompts/tools layer
15. Multi-modal context



The must-have MVP is not “connect everything.” The must-have MVP is **control the loop**. Once the loop is clean, every connector becomes a plugin instead of a mess.